JAGAT GURU NANAK DEV PUNJAB STATE OPEN UNIVERSITY, PATIALA

DEPARTMENT OF COMPUTER SCIENCE & APPLICATIONS

M.Sc. (Computer Science) — Semester-IV

MAJOR PROJECT VIVA VOCE DEFENSE & EXAMINER Q&A GUIDE

Course Code: MSCS-4-03P | Major Project (M.Sc. Computer Science, Semester-IV)

Project Title: Design and Implementation of a High-Throughput, Offline-First, Multi-Tenant SaaS Enterprise POS Architecture with Integrated AI Copilot and Computer Vision OCR
Target Evaluator: External University Examiner & Faculty Defense Committee (JGND PSOU)
Viva Voce Duration: 7 to 10 Minutes

🎖️ Top 15 Tough Technical Questions & Architect's Model Answers

Question 1: "What is the single biggest architectural innovation in your project compared to commercial POS systems like Toast, Square, or Petpooja?"

Architect's Defending Answer:

> *"Respected Examiner, the single biggest innovation is our **True Zero-Wait Hybrid Architecture**. Most commercial POS systems fall into one of two flawed extremes: either they are cloud-dependent web apps that freeze when the network fluctuates, or they are legacy desktop apps with isolated local databases that cannot synchronize real-time kitchen screens across mobile devices.*
> *Our architecture combines the best of both worlds: we treat browser-level IndexedDB as the primary transactional write target ($< 0.5\text{ms}$ write time), giving cashiers unbreakable offline speed, while our decoupled background worker achieves eventual consistency with*

> *PostgreSQL over the cloud, enabling real-time WebSocket kitchen dispatch and centralized multi-branch analytics."*

Question 2: "Explain exactly how PostgreSQL Row-Level Security (RLS) protects tenant data, and why it is superior to application-layer filtering."

Architect's Defending Answer:

> *"In traditional SaaS systems, multi-tenancy is enforced at the application layer—meaning every developer must remember to include* `WHERE tenant_id = current_tenant` *in every ORM query. If a developer forgets this in a single reporting route, cross-tenant data leaks occur.*
> *In SSR ONE AI, we enforce security at the PostgreSQL database kernel:*
> *1. We run* `ALTER TABLE orders ENABLE ROW LEVEL SECURITY;`.
> *2. We define a policy:* `CREATE POLICY tenant_isolation ON orders USING (tenant_id = current_setting('app.current_tenant_id')::bigint);`.
> *3. When a request arrives, our FastAPI dependency extracts the cryptographically verified tenant ID from the JWT and runs* `SET LOCAL app.current_tenant_id = '...'` *on the database connection.*
> *Even if someone executes raw SQL like* `SELECT * FROM orders;`*, the database engine itself physically filters out any row belonging to other tenants. It is mathematically impossible to leak data across tenants."*

Question 3: "How do you guarantee that two cashiers taking orders at the exact same millisecond don't get the same daily sequence number like `#042`?"

Architect's Defending Answer:

> **"We solve this using* ***PostgreSQL Exclusive Row Locking*** *(`SELECT ... WITH FOR UPDATE`).**
> *In our* `daily_order_sequences` *table, we maintain a composite unique constraint on* `(tenant_id, branch_id, sequence_date)`*. When an order commits, the transaction queries this row with* `with_for_update()`*. This places an exclusive lock on that specific counter row.*
> *If Cashier A and Cashier B submit at the exact same microsecond, Cashier A's transaction increments the counter to* `#042` *and commits. Cashier B's transaction is queued in memory for 2 milliseconds until the lock releases, immediately receiving* `#043`*. In our stress tests with 500 simultaneous requests, zero duplicates or gaps were generated."*

Question 4: "Why did you use FastAPI and ASGI instead of Django, which has a built-in admin panel and mature ecosystem?"

Architect's Defending Answer:

> *"Django is fundamentally built on the synchronous WSGI (Web Server Gateway Interface) standard. In WSGI, each incoming request consumes an entire operating system thread. During dinner rushes when hundreds of diners scan QR codes while kitchen printers are active, thread pools exhaust rapidly, leading to high memory overhead and connection timeouts.*
> *FastAPI is built on ASGI and Python's* `asyncio` *event loop. It uses non-blocking coroutines, allowing a single worker process to handle tens of thousands of concurrent connections. Furthermore, FastAPI's tight integration with Pydantic V2 provides compiled C-level schema validation, automatic OpenAPI documentation, and native async database drivers (* `asyncpg` *), giving us the performance of Go or Node.js with the rich machine learning and data science ecosystem of Python."*

Question 5: "If a cashier's terminal is offline for 2 hours and takes 100 orders, walk me through the exact synchronization process when Wi-Fi returns."

Architect's Defending Answer:

> *"1. While offline, all 100 orders are written to local IndexedDB via Dexie.js with* `synced: false` *and client-generated UUIDv4 idempotency keys. The cashier experiences zero delay.*
> *2. When the device reconnects, our* `useZeroWaitOrderSync` *hook detects the browser's* `online` *event or periodic ping heartbeat.*
> *3. The background worker queries un-synced orders in IndexedDB and drains them in concurrent batches via* `POST /api/v1/orders` *.*
> *4. On the server, each order passes through our idempotency cache. If an order was partially received earlier, it is acknowledged without duplicate database insertion.*
> *5. PostgreSQL generates the official daily tokens and commits the transactions.*
> *6. The server returns HTTP 201 Created; the client marks* `synced: true` *in IndexedDB. In our benchmarks, 150 buffered orders synchronized completely in 4.2 seconds."*

Question 6: "How does your real-time Kitchen Display System (KDS) work, and how does it prevent zombie WebSocket connections?"

Architect's Defending Answer:

> *"The KDS operates over full-duplex WebSockets managed by* `KDSConnectionManager` *in our backend.*
> *1. When a kitchen iPad connects, it authenticates with a JWT token and registers under* `branch_id` *and specific kitchen stations (e.g., Tandoor, Curry, Bar).*
> *2. When an order commits, our internal event bus calls* `broadcast_to_branch()`*, pushing the ticket payload to the kitchen in $< 25\text{ms}$.*
> *3. To prevent zombie connections caused by kitchen tablets going to sleep or losing Wi-Fi, the server and client exchange periodic Ping/Pong heartbeat frames every 30 seconds. If a pong is missed, the server cleanly cleans up the socket memory, and the client automatically attempts reconnection with exponential backoff."*

Question 7: "Why did you use localized Tesseract OCR instead of Google Cloud Vision API or AWS Textract?"

Architect's Defending Answer:

> *"Two critical reasons:* ***Operating Cost*** *and* ***Offline Autonomy****.*
> *1. Cloud vision APIs charge approximately $1.50 per 1,000 scans. For a restaurant chain scanning multiple vegetable, dairy, and meat vendor receipts daily across 50 branches, cloud API costs compound significantly.*
> *2. More importantly, cloud APIs fail completely when the restaurant's internet is degraded.*
> *By compiling Tesseract OCR and OpenCV directly inside our Docker backend container, OCR image binarization and text extraction run locally on our own compute infrastructure with* ***zero recurring per-page API fees****."*

Question 8: "How does your AI Copilot avoid hallucinating false numbers when asked about financial metrics?"

Architect's Defending Answer:

> *"We utilize* ***Retrieval-Augmented Generation (RAG) with Dynamic Business Context Injection****.*
> *The LLM is never asked to calculate raw revenue from memory. Instead, when an executive asks: 'What was our food cost this week?', our backend first executes an authoritative PostgreSQL aggregation query within that tenant's RLS session. The exact numerical metrics (e.g., total sales, cost of goods sold, top-selling items) are injected directly into the LLM system prompt as structured JSON.*
> *The LLM's role is restricted to analytical interpretation, narrative structuring, and strategy*

> *formulation based solely on verified database numbers. Furthermore, every AI interaction is recorded in `audit_logs` for managerial accountability."*

Question 9: "What database indexing strategies did you employ to ensure queries remain fast as order tables grow into millions of rows?"

Architect's Defending Answer:

> *"We implemented three specific indexing strategies in PostgreSQL:*
> *1. **Multi-Tenant Composite Indexes**: Every single operational index begins with `tenant_id` (e.g., `CREATE INDEX idx_orders_tenant_status ON orders (tenant_id, status)`). This allows the query planner to jump directly to that tenant's B-tree partition.*
> *2. **Partial Indexes for Soft Deletes**: Because financial records are soft-deleted via `is_deleted = true`, we create partial indexes: `WHERE is_deleted = false`. This keeps index size small and avoids indexing historical archived records.*
> *3. **Range Partitioning**: Our high-volume audit trail table, `audit_logs_partitioned`, is partitioned by range on `created_at` (one partition per month), ensuring that query performance never degrades as historical audit logs accumulate."*

Question 10: "How do you handle physical thermal receipt printing without blocking the cashier's screen?"

Architect's Defending Answer:

> *"We decoupled physical printing from the UI thread entirely using `useAsyncPrintQueue`.*
> *In traditional POS systems, printing synchronously blocks the browser window (e.g., standard `window.print()`). If the printer is out of paper or experiencing a paper jam, the entire cash register freezes.*
> *In SSR ONE AI, the moment payment settles, the print payload is formatted into raw ESC/POS byte buffers and dispatched asynchronously to a local network thermal printer daemon. The cashier's terminal is immediately free to serve the next customer while the hardware prints in the background."*

Question 11: "What are the limitations of this project, and what would you improve next?"

Architect's Defending Answer:

> *"From an academic and architectural standpoint, the main limitation is **Multi-Branch Real-Time Stock Reservation during Long Partitions**. If two offline branches attempt to sell the final unit of a shared warehouse item while both are disconnected, the conflict is only detected during reconciliation.*
> *In the future, I plan to integrate Conflict-Free Replicated Data Types (specifically PN-Counters and bounded counter CRDTs) to allow decentralized quota leases across offline branches. Furthermore, I aim to compile SQLite and small language models to WebAssembly directly inside the browser using WebGPU."*

Question 12: "Did you follow a standard software engineering methodology during development?"

Architect's Defending Answer:

> *"Yes, respected examiner. We followed the **Iterative Spiral / Agile Methodology**. We proceeded from mathematical formalization and database schema design, to Service-Repository pattern implementation, monorepo package encapsulation, and finally automated benchmarking with Locust and Playwright. Every major design decision was formalized in Architectural Decision Records (ADRs) within the project's governance repository."*

Question 13: "How does your architecture map to Brewer's CAP Theorem and Abadi's PACELC Theorem?"

Architect's Defending Answer:

> *"Under Eric Brewer's CAP theorem, any distributed data store can satisfy at most two of Consistency (C), Availability (A), and Partition Tolerance (P). Because network partitions (P) are physical inevitabilities in commercial environments, our architecture deliberately chooses **AP (Availability over Linearizability)** during active partitions.*
> *Under Daniel Abadi's PACELC extension: If there is a Partition (P), how does the system trade off Availability (A) and Consistency (C); Else (E), how does the system trade off Latency (L) and Consistency (C)?*
> *SSR ONE AI maps explicitly to **PA/EL**: During partitions, we prioritize Availability (A) by committing writes to client-side IndexedDB. In the normal connected state (Else), we prioritize Latency (L) by achieving sub-millisecond local UI interaction ($< 0.5\text{ms}$) while deferring server-side synchronization asynchronously."*

Question 14: "If a cashier is offline for 2 hours and their JWT access token expires after 60 minutes, does the terminal lock them out during a dinner rush?"

Architect's Defending Answer:

> *"No, respected examiner. This is a critical edge case in distributed offline systems.*
> *1. **Local Authentication Continuity**: When a cashier logs in, the client securely stores the signed JWT and cryptographic session claims in an encrypted IndexedDB vault (`auth_tokens`).*
> *2. **Grace-Period Offline Authorization**: The client-side security guard verifies the digital signature locally. If the clock exceeds the token's `exp` claim while the device is in an active offline state (`navigator.onLine === false`), the system transitions to an **Offline Grace Period Mode**, allowing continuous cashier billing.*
> *3. **Re-Authentication on Reconnection**: When connectivity returns, the background sync worker uses the stored long-lived HTTP-only Refresh Token to negotiate a fresh JWT before draining the buffered order queue to the server. The cashier's billing flow is never interrupted."*

Question 15: "How do you execute Alembic schema migrations on tables that have active Row-Level Security without causing table-lock deadlocks?"

Architect's Defending Answer:

> *"Executing DDL (`ALTER TABLE`) on tables with active RLS can cause severe exclusive table locks (`AccessExclusiveLock`) that block all incoming cashier transactions.*
> *We solve this through our **Zero-Downtime Migration Standard** (governed by ADR-0007):*
> *1. Schema migrations are executed during maintenance windows via Alembic using `BYPASSRLS` superuser connections, preventing policy evaluation overhead during DDL.*
> *2. Column additions are always defined with `nullable=True` or `DEFAULT` values without rewriting whole tables.*
> *3. Heavy index creations are always executed using PostgreSQL's `CREATE INDEX CONCURRENTLY`, acquiring only `ShareUpdateExclusiveLock` so read and write transactions*